

SIMATS ENGINEERING

CSA1106 - OBJECT ORIENTED ANALYSIS AND DESIGN

Assessment Tool 2

Case Study Solutions

Student Name : KAUSHIK NARAYANAN V

Register Number : 192321047

Department : B.Tech Information Technology

Faculty : Dr. K. Thinakaran

Academic Year : 2023 – 2027

TABLE OF CONTENTS

Question 1: Online Hotel Reservation System	3
Question 2: University Management System	12
Question 3: Online Retail Shopping Platform	21
Question 4: Smart Waste Management System	30
Question 5: Smart Healthcare Management System	39
Question 6: Smart Library Management System	48
Overall Conclusion	57
References	59

QUESTION 1: ONLINE HOTEL RESERVATION SYSTEM

1. Introduction

An **Online Hotel Reservation System (OHRS)** is a web-based software application that allows travellers to search, select, and book hotel rooms from any device connected to the internet. The system eliminates the need for telephone or walk-in bookings by providing a centralised digital platform where guests can compare room types, check availability in real time, view pricing, make secure payments, and receive instant booking confirmations. From the hotel management perspective, the system automates room inventory management, generates occupancy reports, and integrates with third-party payment gateways and travel portals. Using Object-Oriented Analysis and Design (**OOAD**), the system is modelled as a collection of interacting objects — **Guest, Room, Booking, Payment, and Notification** — each encapsulating its own state and behaviour.

2. Problem Analysis

2.1 Real-World Problem

- Manual reservation processes lead to overbooking due to lack of real-time inventory synchronisation.
- Guests cannot verify room availability without contacting hotel staff directly.
- Payment handling is error-prone with no automated confirmation mechanism.
- No centralised reporting for hotel administrators to monitor occupancy trends.
- Cancellation and refund processes are slow and lack transparency.

2.2 Expected Solution

The OHRS addresses these limitations by providing a fully automated, real-time digital platform. It maintains a live database of room availability, automates payment processing, sends instant email and SMS confirmations, supports online cancellation with automated refund computation, and generates analytics dashboards for hotel management. The system supports multiple user roles including **Guest, Hotel Administrator, and Receptionist**, each with role-specific access.

3. Object-Oriented Analysis

3.1 Actors

- **Guest:** Searches rooms, makes reservations, cancels bookings, and makes payments.
- **Hotel Administrator:** Manages room inventory, views reports, and updates pricing.
- **Receptionist:** Handles walk-in reservations and check-in/check-out operations.
- **Payment Gateway:** External system that processes credit/debit card transactions.
- **Notification Service:** Sends confirmation emails and SMS alerts.

3.2 Key Classes and Attributes

Class	Key Attributes	Key Methods	Responsibility
Guest	guestID, name, email, phone, password	register(), login(), searchRoom(), makeBooking()	Represents the end-user who reserves rooms
Room	roomID, type, price, status, floor, amenities	checkAvailability(), updateStatus(), getDetails()	Models a physical hotel room and its properties
Booking	bookingID, guestID, roomID, checkIn, checkOut, status	create(), cancel(), getConfirmation(), computeTotal()	Manages the lifecycle of a reservation
Payment	paymentID, bookingID, amount, method, status, timestamp	process(), refund(), generateReceipt()	Handles all financial transactions
Notification	notifID, recipient, message, channel, sentAt	send(), retry(), log()	Delivers confirmations and alerts to users
Admin	adminID, name, role, permissions	manageRooms(), generateReport(), updatePricing()	Oversees hotel operations and system configuration

4. Object-Oriented Principles

4.1 Encapsulation

Encapsulation is the mechanism of bundling data (attributes) and behaviour (methods) within a class while restricting direct external access to internal state. In the OHRS, the **Payment** class encapsulates sensitive data such as **cardNumber**, **cvv**, and **transactionID** as private attributes. External classes can only trigger payment through the public method **process()**, while the internal payment logic and sensitive data remain hidden. Similarly, the **Room** class encapsulates its availability status and exposes only **checkAvailability()** to booking logic, ensuring data integrity.

4.2 Inheritance

Inheritance allows a child class to inherit properties and methods from a parent class, promoting code reuse. In the OHRS, a base class **User** contains common attributes such as **userID**, **name**, **email**, and **password** along with shared methods **login()** and **logout()**. The subclasses **Guest**, **Administrator**, and **Receptionist** inherit from **User** and extend it with role-specific attributes and methods. For example, **Guest** adds **makeBooking()** and **cancelBooking()**, while **Administrator** adds **generateReport()** and **manageRooms()**. This design eliminates code duplication and simplifies maintenance.

4.3 Polymorphism

Polymorphism enables a single interface to represent multiple forms of behaviour. In the OHRS, the method **sendNotification()** is declared in the abstract **Notification** class and overridden in concrete subclasses **EmailNotification** and **SMSNotification**. When a booking

is confirmed, the system calls **sendNotification()** on whichever notification object is active — without knowing whether it will send an email or an SMS. This allows the system to support new channels (e.g., WhatsApp, Push **Notification**) without modifying the core booking logic. Method overloading is also used in the **searchRoom()** method, which accepts different parameter combinations (date only, date + type, date + price range).

4.4 Abstraction

Abstraction hides implementation complexity and exposes only essential features. The **IPaymentGateway** interface defines abstract methods such as **processPayment()** and **refundPayment()** without specifying how they are implemented. Concrete classes like **StripeGateway** and **PayPalGateway** implement this interface. The Booking module interacts only with **IPaymentGateway**, remaining independent of any specific payment provider.

4.5 Association, Aggregation, and Composition

- Association: A Guest is associated with multiple Bookings over time (one-to-many). A Booking is associated with a Payment (one-to-one).
- Aggregation: A Hotel aggregates many Rooms. If the Hotel object is deleted, the Room objects can independently exist in a historical database.
- Composition: A Booking composes BookingDetail objects (room-night entries). If the Booking is deleted, its BookingDetail records are also removed.

5. Class Identification Summary

Class	Type	Key Attributes	Key Methods
User (abstract)	Abstract	userID, name, email, password	login(), logout(), updateProfile()
Guest	Concrete (extends User)	loyaltyPoints, preferences	searchRoom(), makeBooking(), cancelBooking()
Administrator	Concrete (extends User)	department, accessLevel	generateReport(), manageRooms(), updatePricing()
Room	Entity	roomID, type, price, floor, amenities, status	checkAvailability(), updateStatus(), getDetails()
Booking	Entity	bookingID, guestID, roomID, checkIn, checkOut	create(), confirm(), cancel(), computeTotal()
Payment	Service	paymentID, amount, method, timestamp, status	process(), refund(), generateReceipt()
Notification	Abstract Service	notifID, recipient, message, sentAt	send(), retry(), log()
EmailNotification	Concrete (extends Notification)	subject, emailBody	send() [overridden]

SMSNotification	Concrete (extends Notification)	phoneNumber, smsText	send() [overridden]
-----------------	---------------------------------	----------------------	---------------------

6. Class Relationships

Relationship Type	Classes Involved	Explanation
Association	Guest ↔ Booking	A Guest places one or more Bookings (1..*). Each Booking belongs to exactly one Guest (1).
Generalization	User → Guest / Administrator / Receptionist	User is the parent class; Guest, Administrator, and Receptionist are specialisations inheriting shared attributes.
Dependency	Booking → IPaymentGateway	Booking depends on the IPaymentGateway interface to process payments during confirmation.
Aggregation	Hotel ◇— Room	A Hotel has-a collection of Rooms. Rooms can exist independently of the Hotel entity.
Composition	Booking ◆— BookingDetail	BookingDetail objects are composed within a Booking. They cannot exist without the parent Booking.

7. Suggested UML Diagrams

7.1 Use Case Diagram

The Use Case Diagram for the OHRS includes actors: Guest, Administrator, Receptionist, and external Payment Gateway. Use cases include: Register/Login, Search Rooms, Make Booking, Cancel Booking, Process Payment, Generate Receipt, Send Notification, View Reports, Manage Rooms, and Update Pricing. Guest interacts with Search, Book, Cancel, and Pay. Administrator interacts with Manage Rooms, View Reports, and Update Pricing. The Payment Gateway is an external system participating in the Process Payment use case via include relationship.

7.2 Class Diagram

The Class Diagram shows the inheritance hierarchy (User → Guest, Administrator, Receptionist), the composition between Booking and BookingDetail, aggregation between Hotel and Room, association between Guest and Booking (1 to many), association between Booking and Payment (1 to 1), and the IPaymentGateway interface implemented by StripeGateway and PayPalGateway. All attributes are marked with visibility (+/-/#) and data types.

7.3 Activity Diagram

The Activity Diagram models the room booking process: Guest logs in → Searches available rooms → Selects room → Enters check-in/check-out dates → System checks availability → If available: Guest confirms → Payment gateway processes payment → Booking confirmed

→ Notification sent. If payment fails: Guest is prompted to retry or cancel. If room unavailable: System suggests alternatives.

7.4 Sequence Diagram

The Sequence Diagram for booking creation shows the message flow: Guest → UI: searchRoom() → BookingController: checkAvailability() → RoomDB: queryAvailableRooms() ← returns list → Guest: selectRoom() → BookingController: createBooking() → PaymentService: processPayment() → PaymentGateway: charge() ← returns success → NotificationService: sendConfirmation() → Guest.

7.5 State Diagram

The State Diagram for the Booking entity shows states: New → Pending Payment → Confirmed → Checked In → Checked Out → Closed. An alternate path shows: Confirmed → Cancellation Requested → Refund Processing → Cancelled. Transition triggers include: payment success, payment failure, guest check-in, guest check-out, and cancellation request.

7.6 Component Diagram

Components: User Interface (Web & Mobile), Authentication Module, Room Management Module, Booking Engine, Payment Integration Module, Notification Service, Reporting Module, and Database Layer. The Booking Engine depends on the Payment Integration Module and Notification Service. The Reporting Module depends on the Database Layer.

7.7 Deployment Diagram

Deployment nodes: Client Devices (Browser / Mobile App), Web Server (Node.js / Django), Application Server (Booking Engine, Auth Service), Database Server (MySQL), Payment Gateway Server (Stripe API), Notification Server (Firebase / Twilio), and CDN for static assets. Communication between nodes uses HTTPS and REST APIs.

8. SDLC Model Selection

Selecting the right Software Development Life Cycle (SDLC) model is critical for project success. Below is a comparison of relevant models for the OHRS:

SDLC Model	Suitability for OHRS	Key Limitation
Waterfall	Low — Requirements may change; sequential phases are too rigid for dynamic booking systems.	No room for iterative feedback from hotel clients.
Prototype	Medium — Useful for demonstrating UI to hotel management early.	Prototypes may mislead stakeholders about final system quality.
Iterative	Medium-High — Allows building core booking features first, then adding payments and reports.	May lack formal testing gates between iterations.
Agile (Scrum)	HIGH — Best choice. Supports changing requirements, sprint-based delivery,	Requires disciplined backlog management and daily standups.

OBJECT ORIENTED ANALYSIS AND DESIGN

	continuous client feedback.	
Spiral	Medium — Suitable if hotel chain has high risk tolerance and large budget.	Overhead of risk analysis in each cycle is costly for small projects.
V-Model	Low — Strict verification phases do not suit the evolving nature of booking UX.	Poor flexibility for mid-project requirement changes.

Recommended SDLC Model: Agile (Scrum)

Agile Scrum is the most suitable SDLC model for the Online Hotel Reservation System for the following reasons: First, hotel requirements evolve continuously — new room types, seasonal pricing, loyalty programmes, and payment methods are frequently added. Agile accommodates such changes within **sprint cycles** without disrupting the overall project. Second, the **product backlog** allows hotel management to prioritise features — for instance, the booking engine can be delivered in **Sprint 1**, payment integration in **Sprint 2**, and reporting in **Sprint 3** — delivering business value incrementally. Third, **continuous integration** and testing within each sprint ensure that the system is always in a shippable state, reducing deployment risk. Fourth, the daily Scrum meetings and **Sprint Reviews** facilitate direct communication between the development team and hotel stakeholders, ensuring the product aligns with business goals. Other models such as Waterfall are rejected because a hotel's operational requirements frequently change after initial specifications are written, making the sequential approach impractical.

9. Advantages

- Eliminates overbooking through real-time room inventory synchronisation.
- Provides 24/7 booking capability without reliance on staff availability.
- Secure payment processing with encrypted transaction handling.
- Scalable architecture accommodates hotel chains with hundreds of properties.
- Automated notifications reduce manual communication overhead.
- Rich reporting module enables data-driven hotel management decisions.

10. Challenges

- High Availability requirement — the system must achieve 99.9% uptime to avoid revenue loss.
- Payment security compliance with PCI-DSS standards requires rigorous testing.
- Integration with third-party travel portals (e.g., MakeMyTrip, Booking.com) is complex.
- Handling concurrent bookings for the same room without race conditions requires careful database locking.
- Multi-currency and multi-language support increases development complexity.

11. Conclusion

OBJECT ORIENTED ANALYSIS AND DESIGN

The Online Hotel Reservation System demonstrates how Object-Oriented principles — **Encapsulation**, **Inheritance**, and **Polymorphism** — can be applied to design a robust, maintainable, and scalable application. The class hierarchy with **User** as the base and specialised subclasses ensures clean separation of concerns. The use of the **Notification** abstraction with overridden **send()** methods exemplifies polymorphism in a real-world context. The **Agile Scrum** model is strongly recommended for its ability to handle evolving requirements, deliver incremental business value, and maintain high software quality through continuous testing and stakeholder collaboration.

Key Takeaways

- Encapsulation protects sensitive payment and guest data from external misuse.
- Inheritance through the User hierarchy eliminates code duplication across three actor types.
- Polymorphism through the Notification class enables extensible communication channels.
- Agile Scrum supports the iterative, stakeholder-driven development ideal for a dynamic booking system.

Summary Table

OOP Concept	Applied Class/Method	Benefit
Encapsulation	Payment (private card data)	Data security and integrity
Inheritance	Guest, Admin, Receptionist ← User	Code reuse, less duplication
Polymorphism	EmailNotification, SMSNotification.send()	Flexible, extensible notifications
Abstraction	IPaymentGateway interface	Decoupled payment integration
Composition	Booking ◆ — BookingDetail	Lifecycle-bound relationship modelling

Real-World Applications

- Booking.com — uses similar reservation and payment abstraction layers.
- MakeMyTrip — employs OO design with polymorphic notification channels.
- Marriott Bonvoy — implements loyalty programme through User subclass extensions.
- OYO Rooms — uses Agile development with incremental feature releases.

QUESTION 2: UNIVERSITY MANAGEMENT SYSTEM

1. Introduction

A **University Management System (UMS)** is a comprehensive enterprise software platform designed to automate and integrate all academic and administrative functions of a higher educational institution. It manages student admissions, course enrolment, attendance tracking, examination scheduling, result processing, fee management, faculty allocation, library operations, and hostel management. Using **Object-Oriented Design (OOD)**, the UMS is modelled as a network of interacting objects representing real-world university entities — **Student, Faculty, Course, Department, Examination, and Fee** — each with clearly defined responsibilities and relationships.

2. Problem Analysis

- Manual admission processes cause delays, errors, and loss of applicant data.
- Paper-based attendance records are prone to fraud and difficult to analyse.
- Separate systems for examinations, fees, and library result in data silos.
- Faculty workload monitoring and timetable management are inefficient.
- Students lack real-time access to results, attendance, and fee status.
- Administrative reports require manual compilation, causing decision delays.

The **UMS** resolves these issues by providing a unified, role-based digital platform that integrates all university operations, enables real-time data access, and supports evidence-based academic governance.

3. Object-Oriented Analysis

3.1 Actors

- **Student:** Enrols in courses, views timetables, checks results, pays fees, borrows books.
- **Faculty Member:** Manages attendance, uploads grades, sets question papers, views assigned courses.
- **Department Head:** Allocates courses to faculty, monitors departmental performance.
- **Administrator:** Manages admissions, generates university-wide reports, handles fee structures.
- **Examination Controller:** Schedules examinations, publishes hall tickets, processes and releases results.
- **Librarian:** Manages book catalogue, issue/return tracking, and overdue fine calculation.

4. Key Classes and Relationships

Class	Key Attributes	Key Methods	Responsibility
Person (abstract)	personID, name, DOB, address, phone	getDetails(), updateDetails()	Common base for all human actors

OBJECT ORIENTED ANALYSIS AND DESIGN

Student (extends Person)	studentID, programme, semester, CGPA	enrolCourse(), viewResult(), payFee()	Academic lifecycle of a student
Faculty (extends Person)	facultyID, designation, qualification	markAttendance(), uploadGrade(), viewCourses()	Teaching and assessment responsibilities
Course	courseID, title, credits, syllabus, deptID	getDetails(), enrolStudents(), assignFaculty()	Represents an academic subject
Department	deptID, name, hodID, facultyList	addCourse(), allocateFaculty(), generateReport()	Organises courses and faculty
Examination	examID, courseID, date, venue, duration	schedule(), publishHallTicket(), processResult()	Manages exam logistics and outcomes
Fee	feeID, studentID, amount, dueDate, status	calculate(), pay(), generateReceipt()	Fee billing and collection
Attendance	attendanceID, studentID, courseID, date, status	mark(), getPercentage(), generateReport()	Tracks student presence per course
Result	resultID, studentID, examID, marks, grade	publish(), getGrade(), computeCGPA()	Stores and computes academic results
Library	libraryID, catalogue, totalBooks	searchBook(), issueBook(), returnBook()	Manages library resources

5. Object-Oriented Principles

5.1 Encapsulation

The Student class encapsulates sensitive data such as CGPA and personal address as private attributes. The Fee class hides payment gateway credentials and transaction details. External modules access student GPA only through the public getGrade() method, ensuring consistent grade computation logic.

5.2 Inheritance

Person is defined as an abstract base class with common attributes and methods shared by all university members. Student, Faculty, Administrator, and Librarian extend Person, inheriting shared properties while adding role-specific behaviour. The Department Head is modelled as a Faculty with elevated permissions, demonstrating single inheritance with extension.

5.3 Polymorphism

The generateReport() method is defined in an abstract Reportable interface. It is overridden in Student (to generate academic transcripts), Department (to generate faculty workload reports), and ExamController (to generate result statistics). The same method call produces different output depending on the object type — a textbook example of runtime polymorphism.

5.4 Abstraction

The IPaymentService interface abstracts fee payment operations. Concrete implementations handle bank transfer, UPI, and credit card payments. The Student module interacts only with IPaymentService.pay(), remaining decoupled from the actual payment mechanism.

5.5 Association

A Student is associated with multiple Courses through enrolment (many-to-many). A Faculty member is associated with one or more Courses (one-to-many). An Examination is associated with exactly one Course (one-to-one).

5.6 Aggregation

A Department aggregates many Faculty members. If the Department is dissolved, Faculty records remain independently in the HR database.

5.7 Composition

An Examination composes multiple ExamSlot objects (defining hall, invigilator, and batch). If the Examination is cancelled, all ExamSlot records are deleted with it.

6. Class Relationships

Relationship	Classes	Multiplicity	Description
Association	Student — Course	M:N	Students enrol in many courses; courses have many students
Association	Faculty — Course	1:N	One faculty teaches one or more courses
Generalization	Person → Student, Faculty	IS-A	Student and Faculty are specialisations of Person
Composition	Examination ◆— ExamSlot	1:N	Exam slots are part of examinations
Aggregation	Department ◇— Faculty	1:N	Department contains faculty members
Dependency	Student ⇝ IPaymentService	Uses	Student module uses payment service interface
Association	Student — Result	1:N	One student has many results over their programme

7. Suggested UML Diagrams

7.1 Use Case Diagram

Actors: Student, Faculty, Admin, Exam Controller, Librarian. Use cases include: Apply for Admission, Enrol in Course, Mark Attendance, Upload Grades, Pay Fee, Schedule Exam, Publish Result, Issue Book, Generate Report, Manage Users. Include relationships connect Pay Fee with Generate Receipt; extend relationships connect View Result with Download Transcript.

7.2 Class Diagram

Shows inheritance hierarchy (Person → Student, Faculty, Admin), composition (Examination → ExamSlot), aggregation (Department → Faculty), and many-to-many association between Student and Course via an Enrolment junction class with attributes: enrolmentDate, status, and grade.

7.3 Activity Diagram

Models the examination result publication process: Exam Controller uploads marks → System validates data → Computes grades using grading formula → Checks pass/fail threshold → Publishes to student portal → Sends result notification → Archives in Result table → Generates statistical report.

7.4 Sequence Diagram

Models the course enrolment process: Student → Portal: requestEnrolment(courseID) → EnrolmentController: checkEligibility() → CourseDB: checkSeats() → If seats available: EnrolmentController: addEnrolment() → Fee: calculateFee() → NotificationService: sendConfirmation() → Student: displaySuccess().

7.5 State Diagram

Models the Student admission lifecycle: Applied → Documents Verified → Merit Listed → Offer Made → Fee Paid → Enrolled → Active Student → Graduated | Withdrawn | Suspended.

7.6 Component Diagram

Components: Admission Module, Course Management Module, Attendance Module, Examination Module, Fee Module, Library Module, Notification Service, Report Engine, and Database Layer. All modules communicate through a central API Gateway.

7.7 Deployment Diagram

Nodes: Student/Faculty Devices (Browser, Mobile App), University Web Server, Application Server (with all modules deployed as microservices), Centralised Database Server (MySQL/Oracle), LDAP Authentication Server, and SMTP Notification Server.

8. SDLC Model Selection

For the University Management System, the **Incremental SDLC** model combined with **Agile** principles is recommended. The UMS is a large-scale system with multiple independent modules (**Admission**, **Examination**, **Fee**, **Library**, etc.), making it ideal for incremental delivery. Each module can be developed, tested, and deployed independently. Universities can begin using the **Admission** module while the **Examination** module is still under development. **Agile** practices within each increment allow requirements to evolve (e.g.,

adding **NEP-compliant** grading schemes) without disrupting other modules. The **Waterfall** model is rejected because UMS requirements — especially for examination and result processing — frequently change with regulatory updates. **Spiral** is considered but rejected due to high cost of maintaining risk analysis cycles for each module.

9. Advantages

- Centralised data management eliminates information silos across departments.
- Real-time dashboards empower administrators to make informed decisions.
- Automated fee calculation and payment reduce manual errors significantly.
- Students gain 24/7 access to results, timetables, and fee receipts.
- Object-oriented design ensures easy addition of new modules without system overhaul.

10. Challenges

- High number of concurrent users during examination result publication may strain the server.
- Data migration from legacy paper-based systems to the new digital platform is complex.
- Regulatory compliance with UGC, AICTE, and NBA norms requires frequent system updates.
- Ensuring data privacy of student records under applicable data protection laws.

11. Conclusion

The **University Management System** is an exemplary application of Object-Oriented Design. The **Person** inheritance hierarchy, module-based composition, and polymorphic report generation demonstrate how OOAD principles produce a maintainable and extensible enterprise system. The **Incremental-Agile** approach ensures that each university function is delivered, tested, and operational independently, reducing risk and delivering value throughout development.

Key Takeaways

- Generalization (inheritance) through **Person** avoids duplication across all university actors.
- Composition models the strong lifecycle dependency between **Examination** and **ExamSlot**.
- Polymorphic `generateReport()` enables unified reporting across heterogeneous modules.
- Incremental SDLC allows module-by-module delivery matching university operational needs.

Summary Table

Module	Primary Class	Key Relationship	SDLC Increment
Admission	Student	Student IS-A Person	Increment 1
Academic	Course, Faculty	Faculty–Course (1:N)	Increment 2
Examination	Examination, Result	Composition with ExamSlot	Increment 3
Fee & Finance	Fee, Payment	Student–Fee (1:N)	Increment 4
Library	Library, Book	Aggregation	Increment 5

Real-World Applications

- SASTRA University ERP — Uses OO design with inheritance-based role management.
- VIT University's VTOP — Incremental portal delivery with modular architecture.
- Anna University's CoE Portal — Polymorphic report generation across departments.
- IIT Bombay's Gymkhana System — Role-based access using abstract Person hierarchy.

QUESTION 3: ONLINE RETAIL SHOPPING PLATFORM

1. Introduction

An **Online Retail Shopping Platform** is an e-commerce system that enables customers to browse product catalogues, add items to shopping carts, apply promotional discounts, process orders, make payments, and track deliveries — all through a unified digital interface. For retailers, the platform provides inventory management, order fulfilment, vendor management, analytics, and customer relationship management (CRM). **Modular software design** using **Object-Oriented** principles ensures that each functional domain — **Product, Cart, Order, Payment, Delivery** — is encapsulated in self-contained modules with well-defined interfaces, enabling independent development, testing, and scaling.

2. Problem Analysis

- Monolithic retail applications cannot scale to handle flash sales with millions of concurrent users.
- Inventory inconsistencies arise when product catalogues are not synchronised in real time.
- Manual order tracking leads to customer dissatisfaction and support overload.
- Limited payment options reduce conversion rates and exclude customers.
- Vendor management through spreadsheets is error-prone and lacks audit trails.
- Lack of analytics prevents personalised recommendation and targeted promotions.

3. Object-Oriented Analysis

Modular OO Design in a Shopping Platform

Modular software design partitions the system into independent, cohesive modules — each handling one functional domain. In OO terms, each module is a **package** of related classes with internal encapsulation and external communication through well-defined interfaces. The shopping platform is decomposed into seven modules:

Module	Classes	Responsibility
User Module	User, Customer, Vendor, Admin	Registration, authentication, profile management, role-based access
Product Module	Product, Category, Review, Inventory	Catalogue management, stock tracking, review collection
Cart Module	Cart, CartItem, Wishlist	Shopping cart operations, wishlist management, quantity updates
Order Module	Order, OrderItem, Invoice	Order placement, status tracking, invoice generation
Payment Module	Payment, Coupon, Refund, IPaymentGateway	Secure payment processing, coupon validation, refund management
Delivery Module	Delivery, DeliveryAgent, TrackingEvent	Shipment assignment, real-time GPS tracking, delivery confirmation
Analytics Module	SalesReport, UserBehaviour, Recommendation	Sales analytics, personalised product recommendations, A/B testing

4. Object-Oriented Principles for Modular Design

4.1 Encapsulation — Module Isolation

Each module encapsulates its internal logic and exposes only a public API. The Payment Module hides gateway credentials, encryption keys, and transaction logs as private members. The Cart Module exposes only `addItem()`, `removeItem()`, and `checkout()` publicly, preventing external classes from directly manipulating cart state. This ensures module-level data integrity and simplifies debugging — a fault in the Payment Module cannot corrupt Cart data.

4.2 Inheritance — Actor and Product Hierarchies

The User base class is inherited by Customer (which adds `wishlist`, `orderHistory`, `loyaltyPoints`), Vendor (which adds `productCatalogue`, `commissionRate`, `bankDetails`), and Admin (which adds `systemAccess`, `reportGeneration`). The Product class is extended by `PhysicalProduct` (with `weight`, `dimensions`, `shippingClass`) and `DigitalProduct` (with `downloadURL`, `licenceKey`, `maxDownloads`), demonstrating how inheritance supports diverse product types without code duplication.

4.3 Polymorphism — Payment and Notification Flexibility

The `IPaymentGateway` interface is implemented by `CreditCardGateway`, `UPIGateway`, `WalletGateway`, and `CODGateway`. The Order module calls `gateway.processPayment()` without knowing the concrete implementation. During checkout, the appropriate gateway is selected based on the customer's preference — a classic example of runtime polymorphism enabling a flexible payment ecosystem. Similarly, the Notification abstract class is overridden by `EmailNotifier`, `SMSNotifier`, and `PushNotifier`, allowing the same `sendOrderUpdate()` call to reach customers on different channels.

4.4 Abstraction — Interface-Driven Module Communication

Each module communicates with others only through interfaces, not concrete classes. The Order Module does not depend on a specific Delivery provider — it only knows the `IDeliveryService` interface with methods `schedulePickup()` and `getTrackingID()`. This makes it trivial to swap delivery partners (e.g., replace BlueDart with FedEx) without touching the Order Module code.

4.5 Association, Aggregation, and Composition

Association: Customer places many Orders (1:N). Order has many Products via `OrderItem` (M:N). Aggregation: Vendor has a collection of Products; Products can exist without the Vendor in a marketplace scenario. Composition: Order composes `OrderItems` — if the Order is cancelled, `OrderItem` records are deleted. Cart composes `CartItems` — if the Cart is

OBJECT ORIENTED ANALYSIS AND DESIGN

abandoned, CartItems are purged. Invoice is composed within Order — it cannot exist independently.

5. Class Identification

Class	Attributes	Methods	Responsibility
Customer	customerID, name, email, address, loyaltyPts	register(), login(), placeOrder(), viewHistory()	End-user shopping and account management
Product	productID, name, price, stock, category	getDetails(), updateStock(), applyDiscount()	Product information and inventory management
Cart	cartID, customerID, items[], totalAmount	addItem(), removeItem(), applyDiscount(), checkout()	Temporary shopping session management
Order	orderID, customerID, items[], status, total	create(), cancel(), track(), generateInvoice()	Manages the customer purchase lifecycle
Payment	paymentID, orderID, amount, method, status	process(), refund(), validateCoupon()	Secure transaction execution
Delivery	deliveryID, orderID, agent, status, ETA	assignAgent(), updateStatus(), getETA()	End-to-end shipment management
Recommendation	userID, viewHistory, purchaseHistory	generateRecommendations(), scoreProducts()	Personalised product suggestion engine

6. Class Relationships

Relationship	Classes Involved	Description
Association	Customer — Order	A customer places multiple orders over time (1:N)
Association	Order — Product (via OrderItem)	Many orders contain many products (M:N via junction)
Generalization	User → Customer, Vendor, Admin	All actors inherit common properties from User
Generalization	Product → PhysicalProduct, DigitalProduct	Different product types extend base Product
Composition	Order ◆— OrderItem	OrderItems cannot exist without their parent Order
Composition	Cart ◆— CartItem	CartItems are destroyed with the Cart session
Dependency	Order ⇝ IPaymentGateway	Order uses payment interface for transaction processing
Aggregation	Vendor ◇— Product	Vendor has a catalogue of products; products can outlive vendor

7. Suggested UML Diagrams

7.1 Use Case Diagram

Actors: Customer, Vendor, Admin, Payment Gateway, Delivery Partner. Use cases: Register/Login, Browse Products, Search Catalogue, Add to Cart, Apply Coupon, Checkout, Process Payment, Track Order, Cancel Order, Write Review, Manage Inventory (Vendor), Generate Sales Report (Admin), Assign Delivery Agent (Delivery Partner).

7.2 Class Diagram

Shows User hierarchy, Product hierarchy, Cart→CartItem composition, Order→OrderItem composition, Order→Invoice composition, Customer→Order association, IPaymentGateway interface with concrete implementations, and IDeliveryService interface with FedEx and BlueDart implementations.

7.3 Activity Diagram

Models the checkout process: Customer adds items → Views cart → Applies coupon → Chooses payment method → Payment gateway validates card → If success: Order created, Inventory updated, Delivery assigned, Confirmation email sent. If failure: Error displayed, Customer prompted to retry.

7.4 Sequence Diagram

Models order placement: Customer → CartController: checkout() → OrderService: createOrder() → InventoryService: reserveStock() → PaymentService: processPayment() → DeliveryService: schedulePickup() → NotificationService: sendConfirmation() → Customer: displayOrderID().

7.5 State Diagram

Order states: Placed → Payment Pending → Payment Confirmed → Processing → Shipped → Out for Delivery → Delivered → Return Requested → Refunded. Alternate: Placed → Cancelled → Refunded.

7.6 Component Diagram

Frontend (React/Flutter), API Gateway, User Service, Product Service, Cart Service, Order Service, Payment Service, Delivery Service, Notification Service, Analytics Service, and a shared Database Layer.

7.7 Deployment Diagram

Client (Mobile/Web), CDN (static assets), API Gateway (AWS ALB), Microservices Cluster (Kubernetes), Product DB (MongoDB), Order DB (PostgreSQL), Cache (Redis), Message Queue (RabbitMQ), Payment API (Razorpay), Delivery API (Shiprocket).

8. SDLC Model Comparison and Recommendation

SDLC Model	Pros for Retail Platform	Cons
Waterfall	Simple documentation and planning	Inflexible; cannot handle changing promotions/features
Prototype	Quick UI demonstration to business owners	May result in throwaway code; poor for backend
Iterative	Core shopping features delivered early	Integration issues between iterations
Agile Scrum	Best fit: sprint-based modular delivery, continuous stakeholder feedback, adaptable to market changes	Requires product backlog discipline
Spiral	Excellent risk management for large platforms	Expensive for small-to-medium retailers
V-Model	Strong verification for payment security testing	Rigid; cannot adapt to changing catalogue requirements

Recommended Model: Agile Scrum with Modular Microservices Architecture

The Online Retail Shopping Platform demands the **Agile Scrum** model because e-commerce requirements change constantly — new payment methods, seasonal promotions, flash sale features, and delivery partner integrations are added frequently. Scrum's two-week sprint cycles allow the team to deliver one module at a time (Cart in **Sprint 1**, Order in **Sprint 2**, Payment in **Sprint 3**, Delivery in **Sprint 4**, Analytics in **Sprint 5**). Each sprint produces a deployable, tested module. The modular OO architecture with clearly defined interfaces means each **microservice** can scale independently — the Product Service can be scaled 10x during a sale while the Analytics Service remains at baseline. Continuous Integration and Continuous Deployment (**CI/CD**) pipelines ensure zero-downtime deployments critical for a live retail platform.

9. Advantages

- Modular architecture enables independent scaling of high-traffic services (e.g., Product, Cart).
- Interface-driven design allows seamless replacement of payment or delivery providers.
- OO inheritance supports rapid addition of new product types without breaking existing code.
- Real-time inventory synchronisation prevents overselling during flash sales.
- Agile delivery enables fast response to market trends and competitor features.

10. Challenges

- Distributed transaction management across microservices (e.g., payment success but inventory not updated) requires Saga pattern implementation.

- Search engine performance (full-text search, filters, faceted navigation) requires Elasticsearch integration.
- Cart session management across devices requires Redis-based distributed session storage.
- Recommendation engine requires machine learning integration, adding AI complexity.

11. Conclusion

The Online Retail Shopping Platform demonstrates that modular software design using Object-Oriented principles is essential for building scalable, maintainable e-commerce systems. **Encapsulation** isolates modules, **inheritance** creates reusable actor and product hierarchies, **polymorphism** enables flexible payment and notification strategies, and **abstraction** through interfaces decouples modules for independent evolution. **Agile Scrum** enables continuous delivery of market-relevant features while maintaining architectural integrity.

Key Takeaways

- Modular OO design enables microservice-based deployment for independent scaling.
- Interface-driven communication decouples modules, enabling provider-agnostic integrations.
- Product inheritance hierarchy accommodates diverse product types without redesign.
- Agile Scrum aligns development pace with the dynamic nature of e-commerce business.

Summary Table

Module	OOD Principle	Key Class	Agile Sprint
User Module	Inheritance	Customer, Vendor ← User	Sprint 1
Product Module	Inheritance + Encapsulation	PhysicalProduct, DigitalProduct	Sprint 2
Cart & Order Module	Composition	Cart ♦ CartItem, Order ♦ OrderItem	Sprint 3
Payment Module	Polymorphism + Abstraction	IPaymentGateway implementations	Sprint 4
Delivery & Analytics	Association + Abstraction	IDeliveryService, Recommendation	Sprint 5

Real-World Applications

- Amazon — Uses modular microservices with polymorphic payment gateways globally.
- Flipkart — Applies Agile Scrum for weekly feature releases and A/B tested UI changes.
- Meesho — Uses OO design with Vendor inheritance for social commerce model.
- Myntra — Employs interface-driven Delivery service with multiple courier partners.

QUESTION 4: SMART WASTE MANAGEMENT SYSTEM

1. Introduction

A **Smart Waste Management System** (SWMS) is an **IoT**-enabled, data-driven platform that automates the monitoring, collection, and reporting of municipal solid waste. Equipped with smart bins containing ultrasonic or load-cell sensors, the system continuously measures fill levels and transmits data to a central server via **MQTT** or HTTP protocols. When a bin reaches a configured threshold (typically 80%), the system automatically triggers alerts, notifies collection staff, and optimises collection vehicle routes using real-time GPS data. The system serves multiple stakeholders — **Citizens** (who report issues), **Collection Staff** (who collect waste), **Municipality Administrators** (who monitor performance), and **IoT Sensors** (autonomous data producers).

2. Problem Analysis

- Traditional fixed-schedule collection causes overflow in high-traffic areas and wasted trips to empty bins.
- No real-time visibility into bin fill levels means reactive rather than proactive management.
- Manual route planning for collection vehicles is inefficient, wasting fuel and time.
- Citizens have no digital channel to report illegal dumping or bin overflow.
- Environmental impact data (methane emission estimates, recycling rates) is not tracked.
- No performance metrics to evaluate collector efficiency or identify problem zones.

3. Object-Oriented Design

3.1 Actors and Objects

Actor / Object	Type	Role in System
Citizen	Human Actor	Reports full bins, submits complaints, views collection schedule
Collection Staff (Collector)	Human Actor	Receives optimised routes, marks bins as collected, reports issues
Municipality Admin	Human Actor	Monitors bin levels, views reports, manages zones and staff
IoT Sensor	Automated Actor	Continuously reads fill level and transmits data to server
Bin	Physical Object / Class	Smart dustbin equipped with IoT sensor; has fill level and status
Collection Vehicle	Physical Object / Class	GPS-equipped truck assigned to routes for waste collection
Route	Logical Object / Class	Optimised sequence of bins for a vehicle's collection run
Notification	Software Object / Class	Alert or message sent to relevant stakeholders
WasteReport	Software Object /	Aggregate data report for administrative analysis

	Class	
--	-------	--

4. Key Classes — Attributes and Methods

Class	Attributes	Methods
Bin	binID, location, fillLevel(%), threshold, status, lastEmptied, sensorID	readFillLevel(), sendAlert(), updateStatus(), getHistory()
IoTSensor	sensorID, binID, sensorType, batteryLevel, lastRead, firmware	readLevel(), calibrate(), sendData(), checkBattery()
CollectionVehicle	vehicleID, driverID, capacity, currentLocation (GPS), status	assignRoute(), updateLocation(), markCollected(), getCapacity()
Route	routeID, vehicleID, binList[], totalDistance, estimatedTime, status	optimise(), assign(), getNextBin(), markComplete()
Collector	collectorID, name, vehicleID, zoneID, currentTask	receiveTask(), updateStatus(), reportIssue(), completeTask()
Citizen	citizenID, name, phone, zone, reportHistory[]	submitReport(), trackRequest(), viewNearbyBins()
Notification	notifID, type, message, recipient, channel, sentAt	send(), retry(), markRead()
WasteReport	reportID, zone, period, avgFillLevel, collectionsCount, recyclingRate	generate(), export(), schedule()
Admin	adminID, name, accessLevel, zone	monitorDashboard(), manageStaff(), configureThresholds(), generateReport()

5. Object-Oriented Principles

5.1 Encapsulation — IoT Data Protection

The IoTSensor class encapsulates raw sensor data and calibration parameters as private attributes. Only the readLevel() public method exposes a validated fill percentage to the system. The internal ADC (Analog-to-Digital Conversion) logic, error correction, and battery monitoring are hidden from external classes. The Bin class encapsulates its threshold configuration — external modules cannot directly modify it; they must use configureThreshold() through the Admin interface.

5.2 Inheritance — User and Notification Hierarchies

A User base class provides common identity attributes for Citizen, Collector, and Admin. The Notification abstract class defines the send() template method and is extended by PushNotification, EmailNotification, and SMSNotification, each overriding send() with channel-specific implementation. The Vehicle class could be extended to ElectricVehicle and DieselVehicle, each with different fuelLevel computation logic.

5.3 Polymorphism — Multi-Channel Notification and Route Optimisation

When a bin overflows, the system calls `notification.send()` on whichever Notification subclass is configured for the recipient. Collectors receive `PushNotification` on their mobile app; Admins receive `EmailNotification` with zone-wide fill-level reports; Citizens receive `SMSNotification` confirming their complaint has been logged. The `Route` class implements the `optimise()` method polymorphically — `DijkstraRoute`, `NearestNeighbourRoute`, and `GeneticRoute` are subclasses implementing different algorithms, allowing the system to select the most efficient routing strategy without changing the caller's code.

5.4 Abstraction — Sensor and GPS Interfaces

The `ISensor` interface defines `readLevel()` and `sendData()` abstractly. Concrete sensors (`UltrasonicSensor`, `InfraredSensor`, `LoadCellSensor`) implement this interface. The GPS module is abstracted through the `ILocationService` interface, allowing integration with different GPS providers (Google Maps, HERE Maps, Ola Maps) without modifying the `Route` or `Vehicle` classes.

5.5 Association, Aggregation, and Composition

Association: `Bin` is associated with `IoTSensor` (one-to-one). `Collector` is associated with `CollectionVehicle` (one-to-one). `Admin` is associated with multiple `Zones`. Aggregation: A `Zone` aggregates many `Bins`. If a `Zone` is reconfigured, `Bins` still exist in the database. Composition: A `Route` composes many `RouteStop` objects (each representing a bin in the route). Deleting a `Route` removes all its `RouteStops`.

6. IoT Integration Architecture

The IoT integration layer consists of four tiers: (1) **Perception Layer** — physical sensors embedded in bins measure fill level using ultrasonic echo-distance or load cells; (2) **Network Layer** — sensors transmit data via **LoRaWAN**, **NB-IoT**, or **WiFi** using the **MQTT** protocol to an IoT broker (e.g., **EMQX** or **AWS IoT Core**); (3) **Application Layer** — the backend server processes incoming **MQTT** messages, updates the `Bin` object's `fillLevel` attribute, checks against threshold, and triggers the notification pipeline; (4) **Presentation Layer** — a real-time dashboard displays bin statuses on a city map using **WebSocket** connections.

7. Suggested UML Diagrams

7.1 Use Case Diagram

Actors: Citizen, Collector, Admin, IoT Sensor, Notification Service. Use cases: Report Overflow Bin, Submit Complaint, View Collection Schedule, Receive Collection Task, Update Collection Status, View Optimised Route, Monitor City Dashboard, Configure Bin Threshold, Generate Zone Report, View Analytics, Trigger Alert (IoT Sensor), Send Notification (System).

7.2 Class Diagram

Shows: User → Citizen, Collector, Admin hierarchy; Bin aggregated by Zone; Bin composed with IoTSensor; Route composed with RouteStop; CollectionVehicle associated with Collector and Route; ISensor and ILocationService interfaces; Notification hierarchy.

7.3 Activity Diagram

IoT Sensor reads fill level every 15 minutes → If fillLevel < 50%: log and wait → If 50-79%: log yellow status → If ≥ 80%: trigger alert → Backend creates Notification → Collector receives task → Route optimised → Vehicle dispatched → Bin collected → Fill level reset to 0% → Collection logged in WasteReport.

7.4 Sequence Diagram

IoTSensor → MQTTBroker: publishData(binID, fillLevel) → BinController: updateFillLevel() → ThresholdChecker: checkThreshold() → [if > 80%] NotificationService: sendAlert(collectorID) → RouteOptimiser: addBinToRoute() → CollectorApp: displayTask(binID, location) → [After collection] Collector: markCollected() → BinController: resetFillLevel() → ReportEngine: logCollection().

7.5 State Diagram

Bin states: Empty → Filling (level 1-49%) → Half Full (50-79%) → Full (80-99%) → Overflow (100% / physically overflowing) → Collection Requested → Being Collected → Empty. IoT Sensor states: Idle → Reading → Transmitting → Acknowledged → Error → Reconnecting.

7.6 Component Diagram

Components: IoT Sensor Firmware, MQTT Broker, Bin Monitoring Service, Route Optimisation Engine, Collector Mobile App, Citizen Mobile App, Admin Web Dashboard, Notification Service, Report Engine, PostgreSQL Database, Redis Cache.

7.7 Deployment Diagram

Smart Bins (embedded sensor + microcontroller + NB-IoT module), MQTT Broker (cloud-hosted), Application Server (Docker containers on AWS/Azure), Database Server (PostgreSQL + TimescaleDB for time-series sensor data), GPS Server (Google Maps API), Mobile App Devices (Collector, Citizen), Admin Web Browser, Notification Server (Firebase, Twilio).

8. SDLC Model Selection

SDLC Model	Suitability	Limitation
Waterfall	Low — IoT requirements evolve as sensor technology and smart city standards change	Cannot handle mid-project sensor protocol changes

OBJECT ORIENTED ANALYSIS AND DESIGN

Agile Scrum	High — Iterative delivery of modules; adapts to evolving IoT specs and city needs	IoT hardware testing delays can disrupt sprint velocity
Spiral	Very High — Best choice; structured risk management for IoT hardware failures, data security, connectivity	Higher cost; suits large municipal contracts
Incremental	Medium-High — Modules (Sensor, Dashboard, Route) delivered incrementally	Integration of hardware-software increments requires careful planning
Prototype	Medium — Useful for demonstrating smart bin dashboard to municipality stakeholders	Not suitable for full-scale production system

Recommended SDLC Model: Spiral Model

The **Spiral Model** is the most appropriate SDLC for the Smart Waste Management System because it integrates risk analysis at every phase — critical for a system that combines IoT hardware, real-time communication, GPS routing, and mobile applications. The four quadrants of the Spiral model map perfectly to SWMS development: (1) **Objective Setting** — define smart bin specifications and API contracts; (2) **Risk Analysis** — assess sensor reliability, NB-IoT coverage gaps, data security vulnerabilities, and GPS accuracy; (3) **Development & Testing** — build and validate each system component (Sensor Firmware, MQTT Layer, Backend, Dashboard, Mobile Apps); (4) **Planning** — review with municipality stakeholders and plan the next spiral. The spiral repeats for each major release: **Spiral 1** (basic monitoring), **Spiral 2** (route optimisation), **Spiral 3** (citizen app and analytics), **Spiral 4** (AI-based prediction). Agile is incorporated as the development methodology within each spiral, giving structured risk management at the macro level and agility at the micro level.

9. Advantages

- Real-time bin monitoring reduces overflow incidents by up to 60%, improving public hygiene.
- GPS-optimised routes reduce fuel consumption and collection vehicle operating costs by 25-40%.
- Digital complaint management increases citizen engagement and service accountability.
- Time-series analytics identify high-waste zones for targeted infrastructure investment.
- Automated notifications ensure collectors respond only when bins genuinely require service.
- IoT abstraction layer enables sensor hardware upgrades without application code changes.

10. Challenges

- NB-IoT or LoRaWAN network coverage may be inconsistent in rural or underground areas.
- Sensor battery life management requires remote firmware update (OTA) capability.
- Data security — encrypted communication and device authentication are mandatory.
- Handling burst data during peak waste periods (festivals, events) without system overload.
- Integrating with legacy municipal ERP systems for billing and compliance reporting.

11. Conclusion

The **Smart Waste Management System** is a compelling application of Object-Oriented System Development combined with **IoT** technology. The OO design — with encapsulated Bin and Sensor classes, polymorphic Notification and Route optimisation, and abstracted **ISensor** and **ILocationService** interfaces — produces a system that is resilient, extensible, and maintainable. The **Spiral Model** provides the risk-aware development framework essential for a system that crosses the hardware-software boundary and serves large-scale municipal operations.

Key Takeaways

- IoT abstraction through ISensor interface allows sensor hardware to evolve without backend changes.
- Polymorphic notifications ensure all stakeholders receive alerts through their preferred channel.
- Spiral SDLC provides structured risk management for IoT-hardware-software integration projects.
- OO design with composition (Route ♦ RouteStop) models the real-world routing lifecycle accurately.

Summary Table

Component	OOP Principle	IoT Element	Spiral Phase
Bin Monitoring	Encapsulation	IoT Sensor, MQTT	Spiral 1
Route Optimisation	Polymorphism (Route subclasses)	GPS, Maps API	Spiral 2
Citizen Reporting	Association	Mobile App, Push Notification	Spiral 3
Analytics & AI	Abstraction (IAalyticsService)	Time-series DB, ML Model	Spiral 4

Real-World Applications

- Pune Smart City — Deployed IoT-enabled dustbins with real-time dashboard monitoring.
- Singapore NEA — Uses GPS-optimised routes for collection vehicles across all districts.

QUESTION 5: SMART HEALTHCARE MANAGEMENT SYSTEM

1. Introduction

A **Smart Healthcare Management System (SHMS)** is a comprehensive digital health platform that integrates patient record management, appointment scheduling, telemedicine, pharmacy management, laboratory reporting, insurance processing, and real-time health monitoring (via wearables) into a unified ecosystem. It serves multiple stakeholders — **Patients, Doctors, Nurses, Pharmacists, Lab Technicians, Insurance Companies, and Hospital Administrators** — each with distinct roles and data access requirements. Using Object-Oriented principles, the SHMS is designed with a strong emphasis on data privacy (**Encapsulation**), role-based specialisation (**Inheritance**), and flexible service delivery (**Polymorphism**).

2. Problem Analysis

- Paper-based patient records cause delays, transcription errors, and are difficult to retrieve.
- Appointment booking by phone leads to double-booking, long queues, and patient dissatisfaction.
- Medication errors occur when prescriptions are handwritten or communicated verbally.
- Lab results are delayed and not integrated with the treating physician's view.
- Patients in remote areas lack access to specialist consultations.
- Insurance claim processing is slow due to manual documentation and approval cycles.
- No real-time monitoring of chronic patients between hospital visits increases readmission risk.

3. Object-Oriented Analysis

Class	Attributes	Methods	Responsibility
Person (abstract)	personID, name, DOB, gender, phone, email, address	getDetails(), updateDetails()	Base entity for all human participants
Patient (extends Person)	patientID, bloodGroup, allergies, EMR, insuranceID	bookAppointment(), viewResults(), requestPrescription()	Manages patient health journey
Doctor (extends Person)	doctorID, specialisation, qualification, licenceNo	viewPatient(), diagnose(), prescribe(), conductVideoConsult()	Clinical diagnosis and treatment
Nurse (extends Person)	nurseID, wardID, shift, certifications	administerMedication(), recordVitals(), assistDoctor()	Patient care and medication administration
Pharmacist (extends Person)	pharmacistID, licenceNo, department	dispenseMedication(), verifyPrescription(), updateInventory()	Pharmacy management and dispensing
Appointment	apptID, patientID,	book(), reschedule(),	Manages patient-

OBJECT ORIENTED ANALYSIS AND DESIGN

	doctorID, dateTime, type, status	cancel(), confirm()	doctor meeting lifecycle
Prescription	prescriptionID, doctorID, patientID, medicineList[], dosage	create(), verify(), dispense(), refill()	Digital prescription management
LabReport	reportID, testID, patientID, labTechID, result, status	upload(), validate(), notify(), archive()	Laboratory test result management
EMR	emrID, patientID, visitHistory[], diagnosis[], medications[]	addEntry(), getHistory(), shareWithDoctor()	Electronic Medical Record storage
Insurance	insuranceID, patientID, provider, coverage, claimHistory[]	submitClaim(), getStatus(), computeCoverage()	Insurance claim and coverage management

4. Object-Oriented Principles

4.1 Encapsulation — Patient Data Privacy

Encapsulation is the most critical OO principle in healthcare due to stringent data privacy regulations (**HIPAA, DPDP Act 2023**). In the SHMS, the **Patient** class encapsulates sensitive **Electronic Medical Records (EMR)**, allergy lists, mental health history, and insurance details as private attributes. These are accessible only through role-validated public methods. For instance, `viewEMR(doctorID)` checks whether the requesting doctor has an active appointment with the patient before returning records. The **Prescription** class encapsulates the medicine composition and dosage as private — only the Pharmacist role can call `verifyPrescription()`, preventing dispensing errors from unauthorised modifications. Similarly, the **Insurance** class hides policy calculation logic — patients interact only with `submitClaim()` and `getStatus()`, while the premium calculation formula remains internal.

4.2 Inheritance — Healthcare Professional Hierarchy

The **Person** abstract class serves as the base for all human actors. First-level subclasses include **HealthcareProvider** (extends **Person**) and **Patient** (extends **Person**). **HealthcareProvider** is further extended by **Doctor**, **Nurse**, **Pharmacist**, and **LabTechnician**. **Doctor** is extended by **GeneralPractitioner**, **Specialist**, and **Surgeon** — each inheriting all **Doctor** attributes and methods while adding specialisation-specific behaviour. For example, **Surgeon** overrides the `scheduleConsultation()` method to include pre-operative assessment steps not required by a **GeneralPractitioner**. This multi-level inheritance hierarchy eliminates duplication of identity management, authentication, and common clinical methods across all healthcare professional types. The **Administrator** class extends **Person** independently, with access to system configuration and reporting without clinical privileges.

4.3 Polymorphism — Flexible Consultation and Notification

Polymorphism enables the SHMS to support multiple consultation modalities through a unified **Consultation** abstract class. Three concrete subclasses override the **conduct()** method: **InPersonConsultation** (**conduct()** — directs patient to OPD room), **VideoConsultation** (**conduct()** — initiates a **WebRTC** video session), and **ChatConsultation** (**conduct()** — opens a secure messaging thread). The appointment booking engine always calls **consultation.conduct()** regardless of the type, and the appropriate modality is selected at runtime based on the patient's and doctor's preference. This makes adding new consultation types (e.g., AR-assisted consultation) trivial — a new subclass is added without modifying the booking engine. **Polymorphism** is also applied in the Notification hierarchy: **AlertNotification**, **ReminderNotification**, and **ReportNotification** each override **generateContent()** and **send()** with appropriate message structure and delivery channel.

4.4 Abstraction

The **ILabService** interface abstracts laboratory operations. Concrete implementations support different laboratory equipment: **PathologyLab**, **RadiologyLab**, and **MicrobiologyLab**. The Doctor prescribes a test through **ILabService.requestTest(testType)**, without needing to know which physical lab will perform it. Similarly, **IPaymentProcessor** abstracts billing across cash, card, UPI, and insurance claim pathways.

4.5 Association, Aggregation, and Composition

- Association: Doctor is associated with many Patients through Appointments (M:N). Patient is associated with Insurance (1:1).
- Aggregation: Hospital aggregates Departments; Departments aggregate Doctors. Removing a Department does not delete Doctor records.
- Composition: Patient composes EMR — the Electronic Medical Record is created with the patient and deleted when patient records are purged. Appointment composes ConsultationNote — notes cannot exist without the appointment.

5. Class Identification Summary

Class	Type	Key Responsibility
Person	Abstract Base	Common identity attributes for all system participants
Patient	Concrete (IS-A Person)	Health management, appointments, prescriptions
Doctor	Concrete (IS-A HealthcareProvider)	Clinical diagnosis, prescription, consultation
Nurse	Concrete (IS-A HealthcareProvider)	Medication administration, vitals recording
Appointment	Entity	Lifecycle management of patient-doctor meetings
Consultation (abstract)	Abstract	Template for all consultation types
InPersonConsultation	Concrete (IS-A	Physical OPD appointment

OBJECT ORIENTED ANALYSIS AND DESIGN

	Consultation)	
VideoConsultation	Concrete (IS-A Consultation)	Telemedicine via WebRTC
Prescription	Entity	Digital prescription with dosage and validity
EMR	Composite Entity	Complete medical history for a patient
LabReport	Entity	Laboratory test results linked to patient and doctor
Insurance	Service Entity	Coverage validation and claim processing

6. Class Relationships

Relationship	Classes	Description
Generalization	HealthcareProvider → Doctor, Nurse, Pharmacist, LabTech	Role-based specialisation of healthcare staff
Generalization	Doctor → GP, Specialist, Surgeon	Medical specialisation hierarchy
Generalization	Consultation → InPerson, Video, Chat	Consultation modality polymorphism
Composition	Patient ◆— EMR	EMR is born and destroyed with Patient record
Composition	Appointment ◆— ConsultationNote	Notes exist only within their appointment
Association	Doctor — Appointment — Patient	Three-way relationship modelling clinical visit
Aggregation	Hospital ◇— Department ◇— Doctor	Hierarchical hospital structure
Dependency	Doctor ⇝ ILabService	Doctor requests tests through lab abstraction

7. Suggested UML Diagrams

7.1 Use Case Diagram

Actors: Patient, Doctor, Nurse, Pharmacist, Lab Technician, Insurance Company, Admin.
Use cases: Register/Login, Book Appointment, Conduct Consultation (in-person/video/chat), Prescribe Medication, Dispense Medication, Request Lab Test, Upload Lab Report, View EMR, Submit Insurance Claim, Process Claim, Generate Hospital Report, Manage Staff.

7.2 Class Diagram

Person → HealthcareProvider → Doctor → GP/Specialist/Surgeon; Person → Patient; Consultation → InPersonConsultation, VideoConsultation, ChatConsultation; Patient ◆— EMR; Appointment ◆— ConsultationNote; Hospital ◇— Department ◇— Doctor; ILabService implemented by PathologyLab, RadiologyLab.

7.3 Activity Diagram

Patient books appointment → System checks doctor availability → If available: confirm booking, send reminder SMS → Patient joins consultation → Doctor reviews EMR → Diagnoses condition → Issues e-prescription → System sends to pharmacy → Pharmacist verifies and dispenses → System updates patient medication history → If lab test required: sends request to ILabService.

7.4 Sequence Diagram

Patient → AppointmentController: bookAppointment(doctorID, dateTime) → DoctorCalendar: checkSlotAvailability() → [available] Appointment: create() → NotificationService: sendReminder(patientID, doctorID) → [during consultation] Doctor: conduct() on VideoConsultation → EMR: addEntry(consultationNote) → Prescription: create(medicineList).

7.5 State Diagram

Appointment lifecycle: Requested → Confirmed → Reminder Sent → In Progress → Completed → Prescription Issued → Follow-up Scheduled | Cancelled | No-Show. Lab Report lifecycle: Requested → Sample Collected → Processing → Result Ready → Reviewed by Doctor → Archived.

7.6 Component Diagram

Patient Portal, Doctor Dashboard, Nurse Station, Pharmacy System, Lab Information System, Insurance Integration, Notification Service, EMR Engine, Billing & Payment Module, Telemedicine Module (WebRTC), Analytics & Reporting Module, Database Layer.

7.7 Deployment Diagram

Patient Mobile App / Web Browser, Doctor Workstation, Pharmacy Terminal, Lab Equipment Interface, Hospital Application Server (HIPAA-compliant), EMR Database (encrypted at rest), Lab Data Server, Insurance Integration Gateway, WebRTC Media Server, Firebase Cloud Messaging.

8. SDLC Model Selection

SDLC Model	Suitability	Limitation
Waterfall	Low — Healthcare requirements evolve with regulations and technology (AI diagnostics, wearables)	Cannot adapt to mid-project changes in medical standards
Prototype	Medium — Useful for validating UI with doctors and patients	Weak for complex backend (EMR, lab integration)
Agile Scrum	High — Iterative delivery of modules; stakeholder collaboration with clinical staff	Security and compliance testing requires additional rigor beyond standard sprints
Spiral	Very High — Risk analysis critical for patient data security, regulatory compliance, and clinical accuracy	Higher overhead; needs experienced risk analysts

V-Model	High — Strong verification and validation aligned with healthcare regulatory testing (FDA, CE)	Inflexible for evolving AI/ML diagnostic features
---------	--	---

Recommended SDLC Model: Spiral Model (with Agile sprints within each spiral)

The **Spiral Model** is most suitable for the Smart Healthcare Management System because patient data security, regulatory compliance (**HIPAA, DPDP**), and clinical accuracy demands rigorous risk analysis at every development phase. Each spiral identifies and mitigates risks: **Spiral 1** addresses data breach risks in EMR design; **Spiral 2** addresses prescription error risks in the pharmacy module; **Spiral 3** addresses **HIPAA** compliance risks in telemedicine; **Spiral 4** addresses insurance fraud risks in claim processing. Agile sprints within each spiral ensure clinical stakeholders (doctors, nurses) provide continuous feedback. The **V-Model** is considered as a secondary option for modules requiring strict regulatory testing (e.g., lab result validation), and its verification activities can be incorporated into the Spiral's testing quadrant.

9. Advantages

- Unified EMR eliminates duplicate tests and medication conflicts, improving patient safety.
- Telemedicine module extends healthcare access to rural and remote populations.
- Automated prescription verification reduces medication dispensing errors significantly.
- Real-time lab result integration enables faster clinical decisions.
- AI-based diagnostic assistance (future enhancement) becomes feasible with structured EMR data.

10. Challenges

- Healthcare data privacy regulations (HIPAA, DPDP Act) require expensive compliance audits.
- Integration with hospital legacy systems (HL7 FHIR, DICOM for radiology) is technically complex.
- Doctor adoption of digital workflows requires comprehensive training programmes.
- High availability (99.99% uptime) is mandatory — system downtime can endanger patient safety.
- Telemedicine quality depends on patient internet connectivity — not universally reliable.

11. Conclusion

The Smart Healthcare Management System illustrates the power of OO principles in a high-stakes domain. **Encapsulation** safeguards patient data at the class level. The multilevel

inheritance hierarchy (**Person** → **HealthcareProvider** → **Doctor** → **Specialist**) produces a flexible actor model. Polymorphism through the **Consultation** hierarchy enables the same booking workflow to support physical, video, and chat consultations seamlessly. The **Spiral Model** with embedded **Agile** sprints provides the risk-managed, iterative development process that healthcare's regulatory and clinical demands require.

Key Takeaways

- Encapsulation through private EMR access methods enforces HIPAA-compliant data privacy.
- Multi-level inheritance (Person → Doctor → Specialist) avoids duplication across 6+ actor types.
- Consultation polymorphism enables the same booking engine to support in-person, video, and chat.
- Spiral SDLC with clinical risk analysis ensures patient safety is built into every development cycle.

Summary Table

OOP Principle	Healthcare Application	Business Benefit
Encapsulation	Private EMR attributes, role-validated viewEMR()	HIPAA compliance, data security
Inheritance	Person → HealthcareProvider → Doctor → Specialist	Extensible actor model, code reuse
Polymorphism	Consultation.conduct() → InPerson/Video/Chat	Flexible consultation modalities
Abstraction	ILabService, IPaymentProcessor interfaces	Decoupled, replaceable service modules
Composition	Patient ◆— EMR; Appointment ◆— ConsultationNote	Lifecycle-accurate data modelling

Real-World Applications

- Apollo Hospitals' EMR System — Uses OO design with role-based inheritance for 15+ staff types.
- Practo — Implements polymorphic consultation (in-clinic, video, chat) on a single platform.
- AIIMS National EMR — Spiral model used for phased rollout with HIPAA compliance audits.
- PharmEasy — Pharmacy module with prescription encapsulation and verification methods.

QUESTION 6: SMART LIBRARY MANAGEMENT SYSTEM

1. Introduction

A **Smart Library Management System (SLMS)** is an integrated digital platform designed to automate all operations of a modern library — including book cataloguing, member registration, book issue and return, reservation management, fine calculation, digital resource management (e-books, journals), and inventory tracking. A smart library may also incorporate **RFID**-based self-checkout kiosks, **QR code** scanning for catalogue lookup, automated fine SMS alerts, and an e-catalogue accessible via a mobile app. Using Object-Oriented principles, the SLMS is modelled as an interconnected set of classes — **Member**, **Book**, **Catalogue**, **Loan**, **Reservation**, **Fine**, and **Notification** — each with clearly defined attributes, methods, and relationships.

2. Problem Analysis

- Manual card-based catalogues are slow to search and difficult to update for thousands of books.
- No automated overdue tracking results in books not being returned on time.
- Fine computation is manual and inconsistent — students often dispute amounts.
- Reserved books are held without notification to the requesting member.
- Library staff spend excessive time on manual inventory counts instead of member assistance.
- Digital resources (e-journals, research papers) are not integrated with physical catalogue.
- No real-time visibility into book availability wastes student time on unproductive visits.

3. Object-Oriented Analysis

3.1 Actors

- **Member (Student/Faculty):** Searches catalogue, borrows/returns books, reserves books, pays fines, downloads e-resources.
- **Librarian:** Manages catalogue, issues/returns books, handles complaints, generates reports.
- **Library Administrator:** Manages member accounts, configures fine rules, purchases new books, generates analytics.
- **RFID/Barcode System:** Automated actor that scans book RFID tags for self-checkout and inventory audit.
- **Notification Service:** Sends due-date reminders, overdue alerts, reservation availability notifications.

4. Class Identification — Attributes and Methods

OBJECT ORIENTED ANALYSIS AND DESIGN

Class	Attributes	Methods	Responsibility
Person (abstract)	personID, name, email, phone, address	getDetails(), updateDetails(), login()	Base class for all library system participants
Member (extends Person)	memberID, memberType, borrowLimit, activeLoans[], fineBalance	searchBook(), borrowBook(), returnBook(), reserveBook(), payFine()	Core library patron operations
Student (extends Member)	studentID, programme, semester, rollNumber	viewCatalogue(), downloadEBook(), checkDueDate()	Student-specific library access
Faculty (extends Member)	facultyID, department, designation, extendedBorrowDays	requestPurchase(), viewResearchJournals(), getExtension()	Faculty-specific privileges and access
Librarian (extends Person)	staffID, designation, assignedSection	issueBook(), returnBook(), manageCatalogue(), sendAlert()	Day-to-day library desk operations
Book	bookID, ISBN, title, author, publisher, edition, genre, copiesTotal, copiesAvailable	getDetails(), checkAvailability(), updateCopies(), getISBN()	Physical book entity management
EBook	ebookID, ISBN, title, downloadURL, fileFormat, licenceCount, accessCount	download(), checkLicence(), revokeAccess()	Digital resource management
Catalogue	catalogueID, books[], eBooks[], lastUpdated	search(), addBook(), removeBook(), updateEntry(), generateInventory()	Centralised library inventory and search
Loan	loanID, memberID, bookID, issueDate, dueDate, returnDate, status	issue(), renew(), return_(), computeFine(), getStatus()	Book borrowing transaction lifecycle
Reservation	reservationID, memberID, bookID, requestDate, expiryDate, status	create(), cancel(), confirm(), notify()	Queue management for unavailable books
Fine	fineID, loanID, memberID, amount, daysOverdue, paymentStatus, paidDate	calculate(), waive(), pay(), generateReceipt()	Overdue fine computation and collection
Notification	notifID, recipient, channel, message, scheduledAt, sentAt	send(), schedule(), cancel(), markRead()	Automated alerts and reminders to members

5. Object-Oriented Principles

5.1 Encapsulation — Fine Calculation Logic

The Fine class encapsulates the fine calculation formula as a private method `computeDailyRate()` that accounts for member type (Student or Faculty have different rates) and book category (reference books carry higher daily fines than general circulation books). External modules can only call the public `calculate()` method, which internally uses `computeDailyRate()`. This prevents inconsistent fine computations across different parts of the system. The Member class encapsulates `fineBalance` and `borrowLimit` as private attributes

— the Librarian can only view these through `getFineSummary()` and cannot directly modify them, ensuring the integrity of library account management.

5.2 Inheritance — Member Type Hierarchy

Person is the abstract base class inherited by Member and Librarian. Member is further specialised into Student and Faculty, each inheriting `borrowBook()`, `returnBook()`, and `payFine()` from Member while adding type-specific attributes: Student has programme and semester; Faculty has department and `extendedBorrowDays` (faculty members may borrow books for 30 days instead of the standard 14). The Faculty class overrides `renewLoan()` to allow an additional renewal cycle not available to students. The EBook class inherits from a Resource abstract class alongside Book, unifying physical and digital resources under a common interface without duplicating title, author, and ISBN attributes.

5.3 Polymorphism — Catalogue Search and Notification Delivery

The Catalogue class implements a polymorphic `search()` method overloaded with different signatures: `search(title)`, `search(author)`, `search(ISBN)`, `search(genre, year)`, enabling flexible catalogue queries without creating separate search methods. Compile-time polymorphism (method overloading) is demonstrated here. At runtime, the Notification hierarchy demonstrates polymorphism — `DueDateNotification`, `OverdueNotification`, `ReservationAvailableNotification`, and `NewArrivalNotification` all inherit from `Notification` and override `generateContent()` and `send()` with channel-specific, context-appropriate messages. The same notification dispatch loop calls `notification.send()` for every pending notification regardless of its type.

5.4 Abstraction — Search and Payment Interfaces

The `ISearchEngine` interface abstracts the book search mechanism. Concrete implementations include `KeywordSearchEngine` (simple title/author search), `FuzzySearchEngine` (tolerates spelling errors), and `ElasticSearchEngine` (full-text, faceted search). The library system calls `ISearchEngine.search(query)` without knowing which engine is active — making it trivial to upgrade the search capability. The `IPaymentService` interface abstracts fine payment across cash, UPI, and online banking.

5.5 Association, Aggregation, and Composition

Association: Member borrows Book through Loan (many-to-many via Loan entity). Member reserves Book through Reservation (many-to-many via Reservation entity). Aggregation: Library aggregates Sections; Sections aggregate Books. Removing a Section reclassifies Books but does not delete them. Composition: Loan composes Fine — a Fine cannot exist without its parent Loan. Catalogue composes Entries — if the Catalogue is rebuilt, all Entries are regenerated. Reservation composes `ReservationHistory` — history records are deleted when the Reservation expires or is fulfilled.

6. Class Relationships

Relationship	Classes	Multiplicity	Explanation
Generalization	Person → Member → Student, Faculty	IS-A	Three-level inheritance for member type specialisation
Generalization	Resource → Book, EBook	IS-A	Unifies physical and digital resources under common base
Generalization	Notification → DueDateNotif, OverdueNotif, ReservationNotif	IS-A	Polymorphic notification types
Association	Member — Book (via Loan)	M:N	A member borrows many books; a book is borrowed by many members over time
Association	Member — Book (via Reservation)	M:N	A member can reserve multiple books; books can be reserved by multiple members in queue
Composition	Loan ◆— Fine	1:0..1	A fine is created and destroyed within the lifecycle of a Loan
Aggregation	Library ◇— Catalogue ◇— Book	1:N	Library has a catalogue; catalogue has book entries; books exist independently
Dependency	Member → ISearchEngine	Uses	Member module uses search interface for catalogue queries
Dependency	Fine → IPaymentService	Uses	Fine module uses payment interface for fee collection

7. Suggested UML Diagrams

7.1 Use Case Diagram

Actors: Student Member, Faculty Member, Librarian, Administrator, Notification Service, RFID System. Use cases: Register as Member, Search Catalogue, Borrow Book, Return Book, Reserve Book, Pay Fine, Download EBook, Renew Loan, View Account, Generate Overdue Report, Add New Book, Remove Book, Manage Members, Configure Fine Rules, Send Notification. Include relationship: Borrow Book includes Check Availability. Extend relationship: Return Book extends by Calculate Fine (if overdue).

7.2 Class Diagram

Person → Member → Student, Faculty; Person → Librarian; Resource → Book, EBook; Catalogue composed of Book entries; Loan associates Member and Book with Fine composition; Reservation associates Member and Book; Notification → DueDateNotification, OverdueNotification, ReservationAvailableNotification; ISearchEngine implemented by KeywordSearch, FuzzySearch, ElasticSearch.

7.3 Activity Diagram

Member searches catalogue → Book found → Check availability → If available: Issue Loan, update Book.copiesAvailable, print receipt, schedule DueDateNotification → If unavailable: Create Reservation, add to wait queue → When copy returned: notify first in queue. On return: Scan book → Check due date → If overdue: Calculate Fine → Fine paid → Update Loan status → Increment copiesAvailable → Archive Loan.

7.4 Sequence Diagram

Member → LibraryPortal: searchBook(title) → ISearchEngine: search(title) → CatalogueDB: query() ← returns results → Member: selectBook() → LoanController: issueLoan(memberID, bookID) → Book: decrementCopies() → Loan: create() → NotificationService: scheduleDueDateAlert() → Member: displayLoanReceipt().

7.5 State Diagram

Book copy states: Available → On Loan → Overdue → Returned → Reserved → Checked Out. Loan states: Active → Renewed → Overdue → Returned (on time) | Returned (overdue, fine calculated) → Fine Paid → Archived. Reservation states: Pending → Fulfilled → Expired | Cancelled.

7.6 Component Diagram

Member Portal (Web + Mobile), RFID Integration Module, Catalogue Management Module, Loan & Return Module, Reservation Module, Fine Management Module, Digital Resource Module, Notification Service, Report Engine, ISearchEngine (Elasticsearch), IPaymentService (UPI/Net Banking), Database Layer (MySQL).

7.7 Deployment Diagram

Member Devices (Browser, Mobile App), RFID Self-Checkout Kiosk, Library Web Server (Nginx), Application Server (Spring Boot / Django), MySQL Database Server, Elasticsearch Server, SMTP/SMS Notification Server (Twilio), Digital Resource CDN (for EBook downloads), Payment Gateway (Razorpay for online fine payment).

8. SDLC Model Selection

SDLC Model	Suitability for SLMS	Key Consideration
Waterfall	Low-Medium — Suitable if library requirements are fully defined upfront (unlikely in practice)	Rigid; no room for adding digital resource features mid-project
Prototype	Medium — Good for demonstrating search UI and loan workflows to librarians	Prototype may mislead stakeholders about backend complexity
Incremental	High — Modules (Catalogue, Loan, Fine, Reservation, EBook) can be built and deployed incrementally	Integration testing between increments must be planned carefully
Agile Scrum	Very High — Best for iterative delivery with librarian and student	Requires strong product ownership to prioritise library features

	feedback in each sprint	
V-Model	Medium — Strong verification for fine calculation accuracy and RFID integration testing	Inflexible for evolving digital resource requirements
Spiral	Medium — Good risk management for RFID integration, but overhead is high for a library system	Cost-effective only for large multi-branch library networks

Recommended SDLC Model: Agile Scrum (Incremental delivery of library modules)

Agile Scrum is the most suitable SDLC for the Smart Library Management System because library feature requirements evolve continuously — digital resource integration, **RFID** self-checkout, mobile catalogue apps, and AI-based book recommendation are features that were not anticipated in traditional library systems but are now expected. Scrum enables the team to deliver core borrowing functionality in **Sprint 1**, reservation management in **Sprint 2**, fine calculation and payment in **Sprint 3**, e-resources in **Sprint 4**, and **RFID** integration in **Sprint 5** — each sprint delivering a usable increment validated by librarians and students. The iterative sprint reviews incorporate direct feedback from library staff, ensuring the system meets actual operational needs rather than assumed requirements. Agile's emphasis on working software over documentation is appropriate for a library that needs to see functional modules before committing to a full institutional rollout. **Waterfall** is rejected because library technology is evolving rapidly (AI cataloguing, digital lending) and requirements cannot be frozen at project inception. **V-Model** is considered but rejected for its rigidity, although its verification activities (especially for fine calculation accuracy and **RFID** correctness) are incorporated as acceptance tests within each Agile sprint.

9. Advantages

- Automated fine computation eliminates disputes and ensures consistent, rule-based penalty application.
- RFID self-checkout reduces queue time and library staff workload significantly.
- Reservation notification system ensures books reach the most interested member first.
- Integrated e-resources provide 24/7 access to digital journals without physical library visits.
- Inventory analytics identify high-demand books for purchase and low-circulation books for deaccession.
- Polymorphic notification system supports SMS, email, and app push across all member types.

10. Challenges

- RFID tag installation across thousands of existing books is time-consuming and costly.

- Maintaining digital resource licences and managing concurrent user limits for e-books is complex.
- Data migration from legacy card-based catalogues requires cleansing and standardisation effort.
- Ensuring system availability during peak exam periods when catalogue access demand spikes.
- Privacy concerns around tracking member reading history must be addressed through clear data policies.

11. Conclusion

The **Smart Library Management System** demonstrates a comprehensive application of Object-Oriented principles in an institutional context. The three-level Member inheritance (**Person** → **Member** → **Student/Faculty**) provides a flexible actor model. **Fine** calculation encapsulation ensures consistent and tamper-proof penalty management. The **Catalogue's** polymorphic **search()** method enables diverse query strategies under one interface. The **Loan-Fine** composition models the real-world dependency accurately. **Agile Scrum's** sprint-based, feedback-driven development model ensures the system evolves with the library's growing needs — from physical book management to a fully integrated smart digital library.

Key Takeaways

- Encapsulation of fine calculation logic ensures consistent, dispute-free penalty enforcement.
- Three-level inheritance (Person → Member → Student/Faculty) models library patron diversity accurately.
- Polymorphic Catalogue.search() enables multiple query strategies through a single interface.
- Loan-Fine composition correctly captures the lifecycle dependency in library borrowing scenarios.
- Agile Scrum enables incremental module delivery, validated by real librarians in sprint reviews.

Summary Table

Class	OOP Principle	Key Method	Agile Sprint
Student, Faculty	Inheritance (from Member)	borrowBook(), payFine()	Sprint 1
Catalogue	Encapsulation + Polymorphism	search(title/author/ISBN/genre)	Sprint 2
Loan + Fine	Composition	return (), calculate()	Sprint 3
EBook, Resource	Inheritance (from Resource)	download(), checkLicence()	Sprint 4
RFID Integration	Abstraction	scan(), selfCheckout()	Sprint 5

	(IRFIDService)		
--	----------------	--	--

Real-World Applications

- OPAC (Online Public Access Catalogue) — Used by IIT libraries with polymorphic search engines.
- KOHA Open-Source LMS — Implements OO modular design with inheritance-based member management.
- Ex Libris Alma — Uses Agile development for incremental feature rollout across 1000+ institutions.
- Digital Library of India (DLI) — Integrates EBook with physical catalogue using Resource abstraction.

OVERALL CONCLUSION

This assessment has comprehensively demonstrated the application of Object-Oriented Analysis and Design (OOAD) principles across six diverse and real-world software systems. Each case study — from a hotel reservation system to a smart waste management platform — reveals how the core OO principles of **Encapsulation**, **Inheritance**, **Polymorphism**, and **Abstraction**, combined with structural relationships of **Association**, **Aggregation**, and **Composition**, produce software designs that are modular, maintainable, extensible, and aligned with real-world entities.

Encapsulation emerged as the foundational principle across all systems — protecting payment credentials in the hotel system, patient medical records in the healthcare system, and fine calculation logic in the library system. **Inheritance** was applied consistently to avoid code duplication — the **User/Person** base class pattern appeared in all six systems, demonstrating its universal applicability in multi-actor software systems.

Polymorphism proved particularly powerful in systems requiring flexible service delivery: the **Notification** hierarchy enabled multi-channel communication in all six systems; the **Consultation** hierarchy enabled telemedicine; and the **Route** hierarchy enabled dynamic waste collection routing. Abstraction through interfaces (**IPaymentGateway**, **ISensor**, **ISearchEngine**, **IDeliveryService**) consistently decoupled modules, enabling technology upgrades without system-wide refactoring.

The UML modelling exercises — spanning **Use Case**, **Class**, **Activity**, **Sequence**, **State**, **Component**, and **Deployment** diagrams — collectively validate that OOAD produces comprehensive, implementation-ready design artefacts. The SDLC analysis concluded that **Agile Scrum** is the optimal model for commercially competitive systems (hotel, retail, library), while the **Spiral Model** is preferred for risk-intensive systems involving IoT hardware, patient safety, and regulatory compliance (smart waste management, healthcare).

In conclusion, **Object-Oriented Analysis and Design** is not merely an academic exercise — it is the professional standard for building software systems that must scale, evolve, and remain correct over years of operation. The principles studied in this course are actively applied in industry-leading platforms including **Amazon**, **Apollo Hospitals**, **Pune Smart City**, and university ERPs worldwide. Mastery of OOAD equips software engineers with the conceptual and practical tools to architect solutions that make a genuine difference in society — from sustainable cities to accessible healthcare.

REFERENCES

- [1] Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The Unified Modeling Language User Guide (2nd ed.). Addison-Wesley Professional.
- [2] Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education Limited.
- [3] Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.
- [4] Larman, C. (2005). Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development (3rd ed.). Prentice Hall.
- [5] Coad, P., & Yourdon, E. (1991). Object-Oriented Analysis (2nd ed.). Prentice Hall.
- [6] Rumbaugh, J., Blaha, M., Premerlani, W., Eddy, F., & Lorensen, W. (1991). Object-Oriented Modeling and Design. Prentice Hall.
- [7] Fowler, M. (2004). UML Distilled: A Brief Guide to the Standard Object Modeling Language (3rd ed.). Addison-Wesley Professional.
- [8] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley Professional.
- [9] Jacobson, I., Booch, G., & Rumbaugh, J. (1999). The Unified Software Development Process. Addison-Wesley Professional.
- [10] Martin, R. C. (2003). Agile Software Development: Principles, Patterns, and Practices. Prentice Hall.
- [11] Beck, K., & Andres, C. (2004). Extreme Programming Explained: Embrace Change (2nd ed.). Addison-Wesley Professional.
- [12] Stevens, P., & Pooley, R. (2006). Using UML: Software Engineering with Objects and Components (2nd ed.). Addison-Wesley.
- [13] Object Management Group (OMG). (2017). Unified Modeling Language Specification, Version 2.5.1. Retrieved from <https://www.omg.org/spec/UML/>
- [14] IEEE Computer Society. (1998). IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998). IEEE.
- [15] United Nations. (2015). Transforming Our World: The 2030 Agenda for Sustainable Development. UN General Assembly Resolution A/RES/70/1.